Comparable-based Fee Market

Dynamic Fees based on “what the market will bear”

Preamble

[TODO: Front matter]

Terminology

The key words "SHOULD", "SHOULD NOT", "RECOMMENDED", and "MAY" in this document are to be interpreted as described in BCP 14[1] when, and only when, they appear in all capitals.

The term “actions” is defined as in ZIP-317[2]

- **expiry_window** = 50 blocks must be same as that other ZIP
- **minimum_fee_floor** = 100 zats
- **priority_multiplier** (multiplier) = 10
- **tip** = chain tip
- **K (block lookback from chain tip)** = 5
- **comps** = comparables

Transaction fields: Actual Orchard v5 transaction field names are used whenever possible.

- **nIssueActions** = transaction field, number of actions as defined by ZIP-317
- **fee** = transaction field, fee paid to the miner as zatoshis
- **nExpiryHeight**

Abstract

This ZIP introduces a dynamic mechanism for transaction fees based on comparables (“comps”) from previous blocks. We maintain the ZIP-317 per-action marginal fee, now based on a median calculation of (transaction fees / transaction actions)

The mechanism ships first as simple node and wallet policy changes, and consensus hardening can follow in a later network upgrade. The design preserves Zcash privacy and adds no explicit on-chain metadata beyond the paid fee.

[1] Information on BCP 14 — "RFC 2119: Key words for use in RFCs to Indicate Requirement Levels" and "RFC 8174: Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words"
[2] https://zips.z.cash/zip-0317

Motivation

The price of the ZEC asset has become more volatile and risen sharply, raising transaction fees along with it to prices users have begun to find questionable. Also, there are new environmental factors such as new levels of adoption (the Zashi effect), and the emergence of Zcash Digital Asset Treasuries which hint at increased volume over the foreseeable future.

There are short-term solutions available, the simplest of which would simply be to lower the ZIP-317 marginal fee, but this may be inadequate over a longer period of time and require periodic tuning. Thus, a dynamic fee calculation is required both to weather short-term volatility but also to adjust the marginal fee dynamically over time.

Privacy Implications

- We can create a market with only 1 bit of privacy.
- The user-facing standard + priority 1 potential bit of privacy. (Goal #1)
    - Trade-off: better user experience
    - Negligible information leakage
- Additionally, this design requires trust in the lightwallet's relay of the fees, which potentially collides with https://zcash.readthedocs.io/en/latest/rtd_pages/wallet_threat_model.html
    - Mitigations:
        - nodes must compute/verify locally; relay only if local reference is within a tolerance band;
        - Encourage wallets to use multiple lightwalletd sources to get quorum

Requirements

A successful design and implementation is one that:

1. Does not unnecessarily disclose information about the user
2. Provides a clear, consistent, user experience
    a. Fees are easy to understand and explain
    b. Fees are kept low and adjust rarely
    c. Transactions are reliable under uncongested network conditions
3. Prioritized higher value transactions when the network is congested
4. Incentivizes well-behaved miners to include legitimate transactions in blocks
5. Prevents miners from gaming the protocol by making users pay more
6. Preventing legitimate transactions requires paying a price higher than legitimate users are willing to pay
7. Is easy to implement and deploy:
    a. Current and planned network upgrades are supported, such as ZIP-235's NSM contributions and ZIP-317's action-based accounting.

    b. Requires little to no consensus changes so that it may be easily and quickly deployed.
    c. Wallet cooperation/coordination is soft-enforceable by planned future consensus upgrades
8. Has a fast “attack” and slow “decay”, as in it rises quickly to respond to congestion and decays slowly back to the floor.

Rationale

Based on public economic data - the fee market itself.

K-block lookback to mitigate re-org risk.

Difficulty calculation = 0.94 correlation coefficient

effective_price_per_action = fee / nIssueActions so that 50 issue transaction isn’t underweighted

Specification

Let K

Node Calculation and Indexer Passthrough

Nodes should calculate two items (which can then be passed through by lightwalletd to wallets):

1. **median_fee_per_action**
    a. Gather transactions from blocks **[tip - k - expiry_window … tip - k]** and calculate an list of **effective_price_per_action = fee / nIssueActions**
    b. Fill the empty space in the blocks with empty “synthetic” actions with **minimum_fee_floor * 2** (like ZIP-317 grace actions)
        i. Alternative / Equivalent:
            1. Let C = block capacity
            2. Let U = used block space
            3. Let q = target quantile (e.g., 0.5).
            4. Compute **q′ = clamp((q·C − (C−U)) / U, 0, 1).**
            5. Fee = **quantile(observed per-action prices, q′)**;
                a. if U=0, return the floor.
    c. Return **median([effective_price_per_action])**
2. **Is_congested**
    a. If every block **[tip - k - expiry_window … tip - k]** contains either enough empty space for an empty action, or transactions with fees lower than **median_fee_per_action** return false. Else, return true.

Note: Wallet threat model, we may require a proof but for now we can ensure that the node agrees on the **median_fee_per_action** (perhaps within 1 power of 10) before relaying transactions.

Wallet Construction

Step	User Actions	Calculation
1.	User defines transaction **actions** as defined in ZIP-317.	**nIssueActions = max(2, count(actions))**
2.	Wallet queries lightwalletd for **median_fee_per_action** and **is_congested**	**Lightwalletd returns median_fee_per_action and is_congested**
4.	User chooses **is_priority** bit. If **is_congested** is true, the wallet suggest that the user select “priority”	**final_fee = median_fee_per_action * (priority_multiplier if is_priority else 1)**
5.	Wallet displays total fee	**total_fee = final_fee * nIssueActions**
6.	User submits transaction to the mempool, wallet sets **nExpiryHeight** on the way out	**Chain_tip < nExpiryHeight < chain_tip + expiry_window**

Benefits to Expiry

- It’s a good idea generally speaking
- Better user experience
    - lower expiry heights could be included sooner
    - Predictability (when transaction will expire)

Block Relay, Miner Transaction Selection, Mempool Eviction

Validate: Miners are trusted to act in their own self interest: sort transactions by fee and include them in the blocks until the blocks are full.

Step	User Actions	Policy (and eventually consensus) Rules
1.	Invalid transactions are evicted from the mempool either by gossip routing filters,	• **Tx MUST have nExpiryHeight**

	miner rejection, or (in a later network upgrade) consensus rules	• **chain_tip < nExpiryHeight < chain_tip + expiry_window**
		• `fee / nIssueActions MUST be`
		  ○ `≥ minimum_fee_floor` zats.
		  ○ a power of 10
		  ○ an integer.

2.	NSM Burn (partly a mitigation) to prevent miner inflation of fees	• `To_nsm = fee * 0.6` • `To_miner = fee * 0.4`

Reference Implementation

TODO: Link to simulator.

Footnote placeholder[345678]

[3] https://github.com/zcash/zcash/issues/3473
[4] https://zips.z.cash/protocol/protocol.pdf
[5] Note: There is no protocol-level “miner tip” in Zcash.
[6] pub const BLOCK_UNPAID_ACTION_LIMIT: u32 = 0; Zcashd
[7] Note: Slightly higher fees that people are willing to pay increase network sustainability by returning zats to issuance.
[8] https://www-users.cse.umn.edu/~odlyzko/doc/paris.metro.pricing.pdf